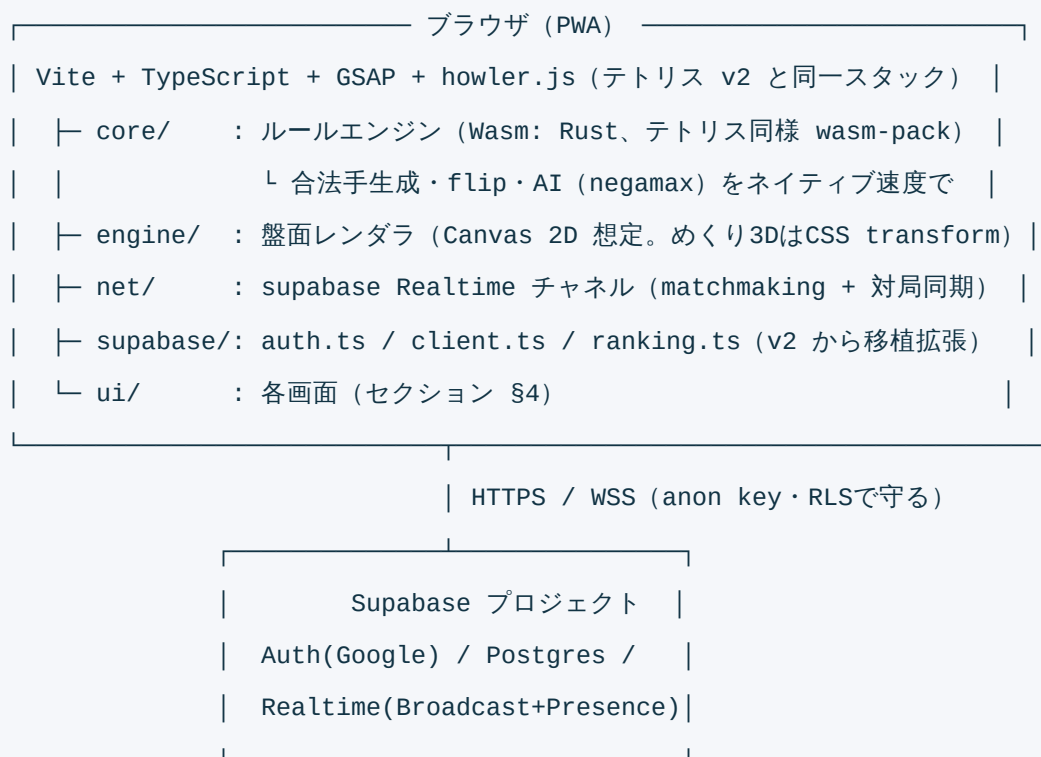


オセロゲーム（仮称：NEON OTHELLO）基本設計書

： v0.9（ドラフト・要件定義書 §8 の回答反映後に v1.0 確定） ： 2026-09-24
 （JST） ： 要件定義書 [2026-09-24_othello-requirements-v0.9.md](#) / 既存資産
[/opt/share/tetris-v2](#)（Supabase + Google OAuth + Vite PWA）

1. 全体アーキテクチャ



静的ホスティング: GitHub Pages (elengine.github.io/neon-othello/)

※ サーバーコード持たず。権限はすべて RLS + リアルタイムポリシーで担保。

- をオセロ用に立てるか、テトリスと同じプロジェクトの別テーブル群にするか → [oth_*](#)。認証設定 (Google OAuth・リダイレクトURL) の追加作業が減り、運用も一本化。GitHub Pages の OAuth リダイレクト先に新URLを追加するのみ。(要確認 Q-8)

- Wasm は任意フェーズで、Round 1 は TypeScript bitboard (Uint32Array×4) でも十分 (60手超でも合法手生成は <1ms、AIは deep 6 まで許容時間内)。テトリスの Wasm 経験は「最強AI」への強化で活かす。

2. データモデル (Postgres / RLS)

2.1 プロフィール拡張 (既存 profiles を継承せず新テーブルで分離)

```
-- oth_profiles: 認証ユーザーの公開情報 + カスタマイズ + レベル
create table public.oth_profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  display_name text not null default '名無し' check (char_length(display_name) <= 20),
  avatar_url text, -- Google icon (null可)
  xp int not null default 0, -- 累計経験値
  level int not null default 1, -- 派生値だが表示性能のため持たせる
  wins int not null default 0,
  losses int not null default 0,
  draws int not null default 0,
  streak int not null default 0, -- 現連勝
  best_streak int not null default 0,
  icon_stones boolean not null default false, -- アイコン石オプトイン (Q-1推奨)
  stone_pref jsonb not null default '{}', -- 石色・盤テーマ設定
  created_at timestampz not null default now(),
  updated_at timestampz not null default now()
);

-- oth_games: 対局の記録 (1対局1レコード・投稿後は不変)
create table public.oth_games (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references public.oth_profiles(id) on delete cascade,
  opp_user_id uuid references public.oth_profiles(id), -- AI戦は null
  mode text not null check (mode in ('ai', 'online', 'local')),
  ai_level int, -- mode='ai' のとき
  result text not null check (result in ('win', 'lose', 'draw', 'abort')),
  black_count int not null default 0,
  white_count int not null default 0,
```

```

moves int not null default 0,                -- 総手数
moves_svg text,                               -- 棋譜(座標列)。将来の再生用
xp_gained int not null default 0,
played_at timestamptz not null default now()
);
-- ※ 1対局で対戦者各自が1レコード投稿する (相互独立・改竄はRLSで自分行のみ)

-- oth_queue: マッチング待機列 (Realtime でローラークラス化)
create table public.oth_queue (
  user_id uuid primary key references public.oth_profiles(id) on delete cascade,
  status text not null default 'waiting' check (status in ('waiting','invited','playing')),
  requested_by uuid,                          -- 招待元 (invited 時にセット)
  updated_at timestamptz not null default now()
);

```

2.2 RLS ポリシー (テトリス v2 と同じ思想)

テーブル	SELECT	INSERT	UPDATE
oth_profiles	全員可 (公開情報)	自分のみ (トリガ自動作成)	自分のみ
oth_games	全員可 (ランキング用)	<code>auth.uid() = user_id</code> のみ	許可しない (不変)
oth_queue	全員可	自分の行のみ	<code>auth.uid() = user_id</code> のみ

- 新規ユーザー作成トリガ `handle_new_othello_user()` で `oth_profiles` を自動行作成 (Google名を初期表示名に、`avatar_url` も `metadata` から)。
- ：クライアントに `xp/wins` の `update` を開けると改竄可能。対策は2案：
- ： `oth_games` `insert` に依存する で `xp/戦績` を再計算・更新 (security definer)。クライアントは対局結果を1回投稿するだけ。
- 案B: Postgres Function (RPC) `submit_game(...)` に集約。→ 検索性は落ちるが改竄ゲートが1つ。 。将来の不正追加 (同じ棋譜の再利用等) も関数内で検証可能。
- リアルタイム接続ポリシー: `realtime.messages` はチャンネル単位 `auth` 確認で十分 (対局チャンネルは参加者2名の `uid` を `broadcast author` に制限)。

- 受諾 → 双方が対局チャンネルへ。 : `oth-match:{game_id}`。
- タイムアウト: `created_at+30s` で `pending` なら `expired` 扱い (クライアント側で無視+定期クリーン)。

3.2 対局同期 (権威サーバーなし・相互検証型)

ターン制のため権威サーバー不要。双方が同一のルールエンジンを持ち、で着手をブロードキャスト:

```
Broadcast event: { t: "move", cell: 0..63, seq: n, ts }
```

- `seq` は 1 から単調増加 (取り決め: 招待側が `seq=0` で "start" を発行、盤面初期化)
- 受信側は手番一致 + 合法手 + `seq` 順序を検証。不整合 → "resync" (手番が棋譜全列を応答)

```
Broadcast event: { t: "pass" } / { t: "resign" }
```

終局: 双方各自が `RPC submit_game()` を呼び記録確定 (結果不一致時は `black/white count` 照合)

- : Presence の退室検知で「切断中」表示 + 猶予60s。復帰時に `oth_games` 未投稿かつ対局中フラグ (`oth_matches` 進行中テーブル、§3.4) から盤面復元 (手番が着手履歴を返答、または進行テーブルのスナップショットを読む)。
- 一手タイマー (Q-5推奨: online 60s): 両クライアントが `ts` 基準で等価にカウント (ズレは `server ts` 基準)、0で `auto-pass` ではなく「時間切れ=着手不能化→2回で終局」を簡潔に:

3.4 進行中对局テーブル (再接続・不正防止のスナップショット)

```
create table public.oth_matches (
  id uuid primary key default gen_random_uuid(),
  game_no bigserial,           -- 対局番号
  black uuid not null, white uuid not null,
  status text not null default 'active' check (status in ('active','ended','abandoned')),
  moves text not null default '', -- 着手座標列 "a1c3..." (復元用・終局後に削除)
  updated_at timestamptz not null default now()
);
-- SELECT当事者のみ / UPDATE当事者のみ / 終局後保持7日 (定期削除)
```

4. 画面設計とレスポンシブレイアウト

4.1 画面一覧

#	画面	内容
S1	タイトル	ロゴ+バージョン、ログイン状態、モード選択（AI戦/オンライン/ローカル/設定）、QR導線
S2	AI難易度選択	レベルカード（見かけ石数で表現）+自分レベル表示
S3	対局画面	盤面+上下HUD（手番・石数・レベルバッジ・タイマー・ミュート）
S4	対戦相手一覧	待機プレイヤーカード（アバター・名前・Lv・勝率・アイコン石可否）、自分の待機ON切替、レベル帯フィルタ
S5	招待応答	モーダル（受諾/拒否、30s カウントダウン）
S6	リザルト	勝敗・石数差・獲得XP・レベルアップ演出（S7）・履歴保存確認
S7	レベルアップ演出	全画面コンフェッティ+称号バッジ拡大+SE（GSAP Timeline）
S8	ランキング	タブ：Lv(XP)/勝利数/勝率。上位100+自分行固定。Lvバッジ・アイコン表示
S9	履歴/戦績	対局リスト+サマリー（勝率・連勝）、フィルタ
S10	設定	石色カスタマイズ（プリセット+カラーピッカー）、アイコン石オプトイン、盤テーマ、名前変更

4.2 レイアウト戦略（回転対応）

CSS: container queries + 論理プロパティ。盤面は正方形：

盤size = min(calc(100vw - 32px), calc(100dvh * 0.60)) ※縦長

横長 (aspect-ratio >= 1.15) → 盤を中央、HUD/情報パネルを左右カラムへ再配置

@media (orientation: landscape) で body class 切替 (touch-device 方式と同様 JS 補完可)

- 縦長：上HUD(相手) / 盤 / 下HUD(自分+操作) – 親指リーチ最優先
- 横長(Fold8展開 4:3)：盤を高く最大化、左右に相手/自分パネル+合法手カウンタ

回転時も盤面scaleのみ変化、DOM/Wasm状態不変（テトリスのちらつき教訓：

overlay は visibility 制御を class 一元管理にし inline style を使わない)

4.3 タッチインタラクション

- 1タップ=着手（プレビューなしで即打ち可能に。誤防は「返す石プレビュー」は長押し0.25sで出る仕様にしてタップと差別化）
- 合法手マーカー：半透明ドット。相手の手番中は盤を触っても反応なし（手番ハイライト矢印）
- めくりアニメ：石を `rotateY(180deg)`（3D transform、方向ベクトルを着手→石の角度に合わせ回転軸を垂直にする）で返る方向を視覚化。多石同時返しも1方向に波及（60ms stagger）。

4.4 石の描画

- Canvas 2D + 事前レンダリング sprites（盤 8×8 描画は軽い）。
- 色石：塗り+グラデ+ネオン外周（テーマ色）。
- `Google avatar` を `drawImage` + 円形クリップで石テクスチャ化（取得は CORS 対応の `lh3.googleusercontent.com`。取得失敗時フォールバック=イニシャル文字）。
- 盤面テーマ：ネオングリーン felt / 和モダン / ダーク木目（プリセット）。

5. AI 設計

レベル	名称	探索	特徴
1	見習い	手数1（ランダム混ぜ10%）	初心者でも勝てる
2	初段	Negamax depth 3 + 角重視	石数の取り合いより角を覚える
3	三段	depth 5、打ち得（frontier）評価	序盤中盤手堅い
4	有段	depth 7 + 終盤完全読み（残り10空→全列举）	上位帯
5	名人	depth 9 反復深化 + 時間予算2s	ほぼ負けない
6	最強(将来)	時間予算5s/Egaroucid移植検討	Round 2

- 評価関数：角重み・辺・X-square回避・フロンティアディスク・手損（odd/even parity、Endgame parity）の重み付け。bitboard（2×uint64相当→Wasm化時は u64）で快速化。
- UIスレッド非ブロッキング：depth ごとの進捗で「思考中…」表示。

6. レベル/XP 設計

獲得XP = 10(着手なし最低) + 1×総手数

+ 対戦種別ボーナス：AI戦 = 5 × AIレベル、オンライン戦 = 20、ローカル戦 = 5

+ 勝敗: 勝利 +20 / 引き分け +5 / 敗北 0 (オンラインは +15/0/-5 の対称調整も検討)

レベル曲線: $Lv\ n$ に必要な累計XP = $50 \times n(n+1)/2$ (Lv1→150, Lv2→450, ... Lv10→2,750)

→ 1日2〜3局で1週間に2〜3Lv上がる緩やかさ。レベル上限 Lv50 (到達は称号「名人」)

称号帯: 1-4 見習い / 5-9 級位者 / 10-19 初段 / 20-29 有段 / 30-39 高段 / 40-49 皆伝 / 50 名人

レベルアップ検知: RPC submit_game 戻り値 { new_level, leveled_to } をクライアントが受け取り S7 演

- : GSAP Timeline - コンフェッティパーティクル (canvas 1レイヤ、80〜150発)、称号バッジ pop-in+シェイク、金ネオンリング拡大、SE (howler、3音階 fanfare)、タイトル画面復帰時にもトースト残置。

7. セキュリティ・不正対策まとめ

1. 戦績更新は RPC othello_submit_game(game) 内のみ (security definer)。クライアントは xp 直接 update 不可 (RLS で oth_profiles の wins/losses/xp 列は update 対象外 → 列レベルポリシー or 別テーブル分離で保証)。
2. 同一棋譜の二度投稿防止: oth_games に (user_id, opp_user_id, moves_hash) 相当の重複チェックを関数内で実施。
3. オンライン結果の相互検証: 双方の投稿石数が食い違う場合、後着のみ採用 (照合不能は記録化して監視)。
4. publishable key は公開前提。ネームスペース oth_* で他アプリと分離。

8. 実装フェーズ

Phase	成果物	見積
P1	リポジトリ scaffolding (Vite PWA / GSAP / howler 移植) + DB SQL (ユーザーがSQL Editor実行)	0.5d
P2	ルールエンジン+UI (S1/S2/S3/S10色石) + AI Lv1-4 (TS実装) + めくりアニメ	2d
P3	Supabase Auth 移植+履歴保存+ランキング (S8/S9) + XP/レベル+演出 (S7)	1.5d
P4	オンライン対戦 (lobby/招待/同期/再接続・タイマー)	2d
P5	レスポンス精緻化 (Fold8実機)、QR配布、デプロイ	1d

9. 追加質問 (設計上の確定事項)

#	質問	ルビー推奨
Q-8	Supabase は新規プロジェクト vs 既存テトリスプロジェクトに oth_* テーブル追加	既存プロジェクト追加（OAuth設定流用・運用一元）
Q-9	ゲーム名（アプリ名・リポジトリ名）は「NEON OTHELLO / neon-othello」でOK？	OK（ブランド一貫）。他案：REVERSI NEON 等
Q-10	石色の初期プリセット（黒白の定番を既定にするか、ネオンテーマを既定にするか）	既定＝ネオン（他と差別化）、定番の黒白もすぐ選べる